

DL066 推导与实验答案

正确答案不应只列出哪个开关更省内存。应说明目标、数值路径、测量对象和适用条件。下面的实验数字来自交付结果，计时只是这次CPU环境的观测。

1 范围与局部刻度

FP16使用5个指数位、10个显式小数位；BF16使用8个指数位、7个显式小数位，二者各有1个符号位。规范化数隐含前导1，因此在1附近分别有11位与8位有效二进制精度。

FP16在1附近的间距为2的负10次方，约0.00097656；BF16间距为2的负7次方，约0.0078125。BF16较粗，却能表示接近FP32的数量级范围。不能简单用“BF16更高精度”概括二者。

1e-8转FP16得到0，70000转FP16得到正无穷；BF16保留这两个量级的近似值。1加2的负10次方在FP16可以表示，在BF16舍入回1。这些例子分别检验下溢、溢出与局部舍入，不能混为同一种错误。

finfo.tiny是最小正规正数。次正规数允许更小的非零数，但相对分辨率与支持行为需要留意。使用tiny断言所有更小数必为0是错误的。

2 两个向量微批的正确归约

有效元素数3与9，总数12。正确权重1/4与3/4，所以梯度为

$$g = \frac{1}{4}(2, -1) + \frac{3}{4}(-2, 3) = (-1, 2).$$

错误等权结果为 $\frac{1}{2}(2, -1) + \frac{1}{2}(-2, 3) = (0, 1)$ 。错误方法把小微批里每个元素的权重设成1/6，大微批每个元素为1/18，前者是后者三倍；真正全体元素均值应每个元素都是1/12。

本课31、47、50微批同理。主要网络每个样本输出1个数，所以按样本数与按MSE元素数相同。若不同样本输出不同有效元素数量，必须明确目标是“每样本等权”还是“每有效元素等权”，两者本来就是不同研究问题。

3 清梯度与更新的正确时机

若每个微批前都zero_grad，前面微批累积的梯度被擦掉，最后只剩最后一个微批的贡献。即使loss乘了正确权重，最终仍不是完整有效批梯度。

若每个微批都step，参数在中途改变。后一个微批计算的梯度来自新参数点，因此不再是同一参数点上全部梯度的和；带动量或Adam时，优化器状态更新次数也改变。

正确顺序是在有效批开始清梯度，各微批反向并累积，结束后执行一次更新。若使用loss scaling，整个有效批的scale应保持一致，不能把不同尺度的梯度混加。每个有效批才执行一次相应的unscale、step与scale更新。

4 为什么BatchNorm破坏朴素等价

BatchNorm训练时使用当前批的均值方差。全批128个样本得到一组统计，拆成31、47、50后得到三组统计。即使每个样本loss最后正确加权，它在前向中经历的函数已经改变，梯度自然不必相同。

因此正确归约是必要条件，不是充分条件。本课BatchNorm相对梯度误差约0.1986，不应被掩盖为浮点噪声。若固定统计或替换归一化方式，可以重新设计对照，但那是修改模型假设，需要明确报告。

同理，随机增强和Dropout的随机样本若不能匹配，也会造成差异。不过概率分布层面的训练可能仍有意义；那与逐步数值等价是两个不同主张。

5 loss scaling的顺序

原梯度 g 为 $1e-8$ ，乘65536后约0.00065536，进入FP16可表示区间。转换后再用FP32除65536，可以得到接近 $1e-8$ 的值。若先转FP16， g 变成0，再做任何有限倍率乘除都仍然是0。

这说明缩放要在低精度反向丢失信息之前生效。它不保证所有梯度都安全：scale过大可能使大梯度溢出，前向已有无穷也无法靠缩放修复。动态scaler的检测、跳步和尺度更新正是为了处理一部分这类问题，而不是保证训练一定收敛。

解除缩放后才按原始梯度单位裁剪。若阈值为1，却对放大65536倍的梯度直接裁剪，等效原始阈值变得极小，更新方向和幅度都会改变。

6 CUDA扩展计划的合格答案

先建立同一数据与模型的FP32 CUDA基线，固定随机种子、有效批大小、更新预算、评价路径和计时边界。再用autocast的FP16策略与GradScaler，监测loss、梯度有限性、scale、跳过更新数和最终指标。

在一次有效批中，逐微批计算按真实归约单位加权的loss，调用`scaler.scale(loss).backward`。全部微批结束后，若要裁剪，先`scaler.unscale_(optimizer)`，再裁剪；最后`scaler.step(optimizer)`和`scaler.update`。对同一个optimizer的一次更新不要重复unscale。

速度和峰值显存要在真实GPU上按065的方法单独测量，包含warmup和同步。当前CPU BF16结果不能支持CUDA FP16加速倍数、Tensor Core利用率或GPU峰值显存结论。计划不是执行证据，没运行就写未运行。

7 简化重算内存模型

用 $M/A = 25/k + k$ ，导数为 $-25/k^2 + 1$ 。令其为0得到 $k=5$ ，简化保存量约10A，而完整保存约25A。二阶导数在正 k 时为正，这一点是极小值。

模型忽略参数、梯度、优化器、输入输出、算子工作区、框架对象、非均匀层大小、分支与分配器缓存。实际整数分段也要处理无法整除的情况。它适合解释边界保存与段内重算之间的平衡，不能充当GPU峰值预测器。

若原前向耗时F、反向2F，总计3F；增加约一次前向，变成4F，理想比例4/3。当前CPU小网络实测约1.85，不矛盾，因为Python与checkpoint机制开销、实际算子比例及非重算head都不满足最简模型。

8 Dropout与同一函数

Dropout会抽取随机mask。非重算前向使用mask A；若反向重算得到mask B，保存时和重算时的函数不同。即使输入和参数相同，梯度也不能保证匹配。

本课在两条路径开始前固定同一CPU随机种子。preserve_rng=True时误差0；关闭保留后约0.5413。这个比较专门隔离随机状态机制，没有把不同模型初始化或数据差异混进去。

使用全局计数器决定分支、重算期间改变训练模式、依赖可变外部状态，也是可能让函数不同的例子。多设备随机状态与函数内设备迁移的保证更复杂，不能从当前CPU例子直接外推。

9 保存量与峰值的区别

完整路径164864字节，重算66560字节，减少98304字节；除以164864得到约59.63%。这个值来自前向阶段hooks看见的非参数唯一storage，含输入和目标。

它没有表示反向重算时的临时峰值，没有包括参数和优化器，也没有衡量分配器保留内存或进程RSS。因此合法结论是“此定义下的前向保留载荷减少约59.63%”。把它改写成“总训练内存下降59.63%”会越过证据边界。

同一存储可能有多个view，按storage去重避免重复计数。反过来，存储中可能只使用一部分view，计storage字节又与逻辑元素字节不同；所以字段命名和scope说明都必须保留。

10 单步结果与训练结果的联合解释

主要模型的FP32参考loss约0.707337。BF16单步相对梯度误差约0.002377，正确累积约 $1.27e-7$ ，重算为0；错误等权约0.05168。前两种非零误差不能一概视为同一问题，错误权重改变目标，BF16改变数值近似。

三种子最终FP32测试MSE约0.124388、0.121315、0.119268；BF16约0.124405、0.121365、0.119318。正确累积在极小容差内与FP32相同，重算本次相同。这支持当前小任务和160次更新范围内的数值兼容性，不构成大规模模型保证。

12个最终权重重新加载后应复现各自统一FP32推理的测试MSE。它们不包含完整优化器和随机状态，不能拿来证明中途断点续训完全一致。实验可复核与严格续训是不同交付要求。

11 一个合理的下一步

若目标是降低真正的GPU峰值，先在GPU上测完整FP32基线，再分别测autocast、微批累积和重算，保持学习目标、有效批大小与更新预算一致。报告allocated/reserved峰值、稳态吞吐和最终模型误差；任何不支持的算子或数值失败都保留。

如果结果出现差异，应先单步比较，再逐层或逐策略定位，而不是同时增大容差和减少训练步数。此时一份负结果也有价值：它告诉你某个节省方法在当前假设下不够稳定，或它节省的并不是实际瓶颈。